Domino: Decoupling Causal Modeling from Autoregressive Drafting in Speculative Decoding

Jianuo Huang[1,2*] **Yaojie Zhang**[1,3*] **Qituan Zhang**[4]
Hao Lin[2] **Hanlin Xu**[5] **Linfeng Zhang**[1†]
[1]EPIC Lab, Shanghai Jiao Tong University [2]School of Software Engineering, HUST
[3]UESTC [4]Fudan University [5]Huawei
[*]Equal contribution. [†]Corresponding author.

Abstract

Speculative decoding accelerates LLM inference by drafting multiple tokens and verifying them in parallel with the target model. However, its practical speedup is constrained by the trade-off between draft quality and drafting cost: autoregressive drafters model causal dependencies among draft tokens but incur sequential overhead, while parallel drafters reduce drafting cost but weaken intra-block dependency modeling. In this paper, we propose Domino, a speculative decoding framework that decouples causal dependency modeling from expensive autoregressive draft execution. Domino first uses a parallel draft backbone to produce preliminary draft distributions for the entire block, and then applies a lightweight Domino head to refine them with prefix-dependent causal information. To stabilize teacher-forced causal encoding, we further introduce a base-anchored training curriculum that first strengthens the parallel backbone and then gradually shifts optimization toward the causally corrected final distribution. Experiments on Qwen3 models show that Domino achieves up to $5.49\times$ end-to-end speedup under the Transformers backend and up to $5.8\times$ throughput speedup under SGLang serving.

Links: Code (GitHub) | Models (Hugging Face)

1 Introduction

While large language models have achieved expert-level performance on reasoning, coding, and long-context tasks (Singh et al., 2025), their standard autoregressive decoding remains inherently sequential. This process is often memory-bound, leaving the massive parallelism of modern GPUs underutilized and leading to high inference latency. To alleviate this bottleneck, speculative decoding has emerged as a widely adopted strategy for accelerating LLM inference (Leviathan et al., 2023). It leverages a lightweight draft model to propose multiple future tokens, which are then verified in parallel by the target model within a single forward pass. By reducing the number of expensive invocations of the target model, this draft-then-verify mechanism preserves the target model's output distribution while improving throughput. Subsequent work has explored various drafting strategies, including head-based multi-token prediction (Cai et al., 2024) and autoregressive draft models (Li et al., 2024b,a, 2025b). Across these methods, the resulting speedup is jointly determined by two factors: *draft quality and drafting cost*. Higher-quality drafts yield longer acceptance lengths, but the drafting process itself introduces additional overhead that can diminish or even negate these gains.

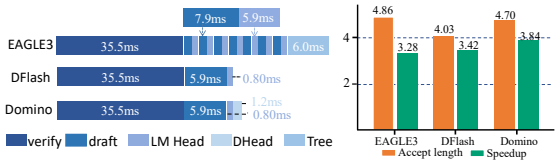


Figure 1: Latency breakdown and performance comparison on Qwen3-8B under a 16-token speculative decoding budget. Left: per-step latency breakdown measured on an A100 GPU with context length 1024, where *Verify* denotes the target-model verification latency and *Draft* denotes the draft-model forward latency. *LM Head*, *DHead*, and *Tree* denote the output projection, Domino head, and tree construction/sampling overheads, respectively. Right: acceptance length and end-to-end speedup evaluated on GSM8K. All three draft models are trained on the same dataset.

This quality–cost trade-off is especially evident in autoregressive drafting methods such as the EAGLE series (Li et al., 2024b,a, 2025b). By generating draft tokens sequentially, autoregressive drafters explicitly model causal dependencies within the draft block, improving alignment with the target model's autoregressive distribution and yielding long acceptance lengths. However, generating k draft tokens requires k sequential draft steps, each involving a draft-model forward pass

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 23346–23356
October 24-29, 2026 ©2026 Association for Computational Linguistics

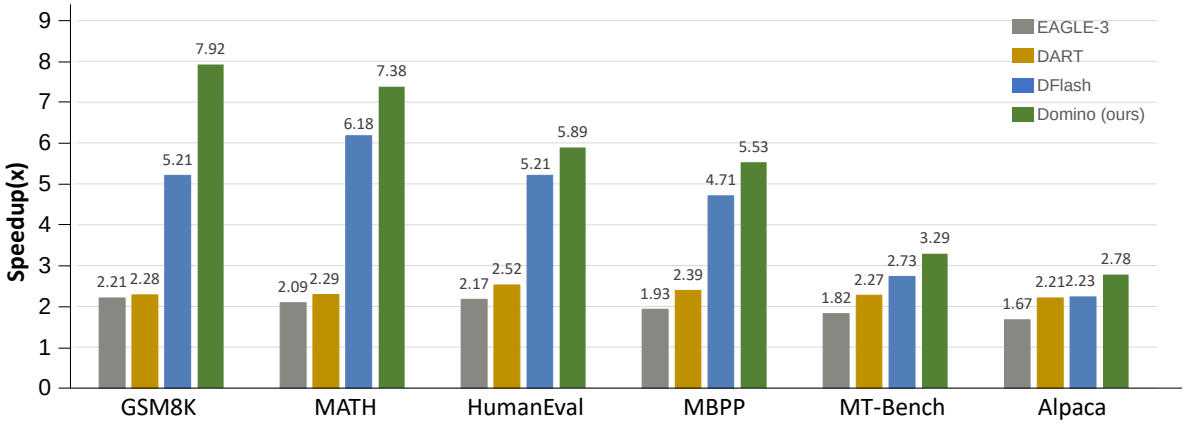


Figure 2: Speedup comparison of Domino, DFlash, and EAGLE-3 relative to autoregressive decoding on Qwen3-8B using the Transformers backend.

and a full-vocabulary LM-head projection. This cost grows linearly with draft length and can offset the gains from higher acceptance length, limiting the speedup obtained by scaling draft length or draft capacity (Zhao et al., 2025; Yan et al., 2025).

Existing methods address this issue from different directions. FR-Spec (Zhao et al., 2025) and SpecVocab (Williams et al., 2026) reduce the cost of full-vocabulary projection through static or dynamic vocabulary selection, but still retain autoregressive draft execution. In contrast, DFlash (Chen et al., 2026) fully parallelizes drafting by producing an entire draft block in one forward pass, avoiding repeated per-token draft-model and LM-head calls. However, removing sequential draft dependencies weakens intra-block causal modeling and can reduce the alignment between the draft distribution and the target model distribution.

Figure 1 illustrates this trade-off. EAGLE-3 obtains a high acceptance length of 4.86, but its sequential draft execution and tree construction limit the speedup to $3.28\times$. DFlash reduces drafting overhead through block-parallel generation and improves speedup to $3.42\times$, but its acceptance length decreases to 4.03. These results suggest that causal dependency modeling is useful for draft quality, but the standard autoregressive implementation makes it expensive. This raises a natural question: can we retain the draft-quality benefit of causal dependency modeling while preserving the low drafting cost of block-parallel generation?

To answer this question, we propose Domino, a lightweight causal correction framework that decouples causal dependency modeling from expensive autoregressive draft execution. Instead of generating draft tokens sequentially, Domino keeps the main drafting computation parallel: a parallel draft backbone first produces preliminary draft distributions for the entire block. On top of these distributions, Domino applies a lightweight Domino head to inject causal information. The Domino head uses a causal encoder to summarize previously drafted tokens and a low-rank correction head to refine the draft distributions through residual correction, avoiding repeated draft-model execution and another expensive full LM-head computation.

This design allows Domino to recover useful intra-block causal dependency while preserving the efficiency of block-parallel drafting. As shown in Figure 1, compared with DFlash, Domino adds only 56M parameters (+5.3%) and incurs only a 2.8% increase in total draft-then-verify latency. Simultaneously, it improves average acceptance length by 16.6% and end-to-end speedup by 12.3%.

Figure 2 provides a preview of the empirical gains. On representative math, code, and chat benchmarks, Domino consistently outperforms EAGLE-3, DART, and DFlash, achieving up to $7.92\times$ speedup on GSM8K and improving over DFlash from $5.21\times$ to $7.92\times$. These results suggest that lightweight causal correction improves draft quality while preserving the efficiency of block-parallel drafting.

2 Related Work

Speculative Decoding. Speculative decoding accelerates autoregressive LLM inference by using a draft mechanism to propose candidate tokens and a target model to verify them in parallel. Early approaches use a smaller language model as the

drafter, which generates candidate tokens autoregressively before verification by the target model (Leviathan et al., 2023; Chen et al., 2023). Subsequent methods improve this basic draft-then-verify pipeline through better draft construction and verification (Miao et al., 2023; Cai et al., 2024; Li et al., 2024b), model-free drafting (He et al., 2024; Liu et al., 2026b), parallel draft-verification execution (Liu et al., 2025; Xie et al., 2026a), and MoE-aware speculative decoding (McDanel et al., 2026; Xie et al., 2026b). These works establish the general framework of speculative decoding, where the final speedup depends on both the acceptance length and the cost of generating draft tokens.

Autoregressive and Efficient Drafting. A representative line of speculative decoding methods improves draft quality through autoregressive drafting. The EAGLE series generates draft tokens sequentially, allowing each token to depend on previous draft tokens and better match the target model's autoregressive distribution (Li et al., 2024b,a, 2025b). Other methods reduce drafting overhead from different angles: Medusa uses lightweight parallel decoding heads (Cai et al., 2024), Hydra introduces sequentially-dependent heads to inject causal information into head-based drafting (Ankner et al., 2024), and FR-Spec and SpecVocab reduce full-vocabulary projection costs through static or dynamic vocabulary selection (Zhao et al., 2025; Williams et al., 2026).

Parallel and Non-Autoregressive Drafting. Speculative Diffusion Decoding first explores discrete diffusion models as parallel drafters for speculative decoding (Christopher et al., 2025). DiffuSpec further uses pretrained diffusion language models as training-free drafters (Li et al., 2025a). PARD adapts autoregressive models into parallel draft models, allowing multiple future tokens to be predicted in a single draft forward pass (An et al., 2026). More recently, DART predicts token distributions for multiple future positions in parallel and uses tree pruning to construct draft candidates (Liu et al., 2026a). DFlash adopts a block-diffusion drafter that produces an entire draft block in a single forward pass, avoiding repeated calls to both the draft model and the full LM head (Chen et al., 2026). These methods substantially reduce drafting overhead, but fully parallel drafting weakens intra-block causal dependencies, making it harder to match the target model's autoregressive distribution. In contrast, Domino keeps the main draft computation parallel while reintroducing causal information through a lightweight correction branch.

3 Preliminaries

3.1 Speculative Decoding and Speedup

Speculative decoding accelerates autoregressive inference by using a draft model M_d to propose multiple future tokens, which are then verified in parallel by the target model M_t. At each decoding cycle, the draft model proposes γ candidate tokens. The target model evaluates these candidates in a single forward pass and accepts the longest valid prefix according to the standard speculative verification rule. We denote by $\tau \in [1, \gamma + 1]$ the expected number of tokens advanced per cycle, including the bonus token produced by the target model.

The average per-token latency of speculative decoding can be written as

$$L_{\text{spec}} = \frac{T_{\text{draft}} + T_{\text{verify}}}{\tau},$$

where T_{draft} is the time spent generating draft tokens, and T_{verify} is the time spent verifying them with the target model. Let L_{target} denote the per-token latency of standard autoregressive decoding with the target model. The resulting speedup is

$$\eta = \frac{L_{\text{target}}}{L_{\text{spec}}} = \frac{\tau L_{\text{target}}}{T_{\text{draft}} + T_{\text{verify}}}.$$

This expression highlights two key factors that determine the final speedup. First, increasing the acceptance length τ allows each target-model invocation to advance more tokens. Second, reducing the draft cost T_{draft} prevents the drafting stage from offsetting the benefit of parallel verification. Therefore, an effective drafter must be both accurate enough to achieve long acceptance lengths and efficient enough to keep drafting overhead low.

3.2 Autoregressive Drafting

Autoregressive drafters generate candidate tokens sequentially. Given a prefix $x_{\leq t}$, an autoregressive drafter factorizes the draft distribution as

$$q_{\text{AR}}(x_{t+1:t+\gamma} \mid x_{\leq t}) = \prod_{i=1}^{\gamma} q(x_{t+i} \mid x_{<t+i}),$$

where $x_{<t+i}$ includes the current prefix $x_{\leq t}$ and previously drafted tokens. This factorization mirrors the target model's autoregressive prediction process, where each token is conditioned on all previous tokens. As a result, autoregressive drafting

can explicitly use previously drafted tokens when predicting later draft positions, leading to higher draft quality and longer acceptance length.

However, this modeling advantage comes with a sequential execution cost. More concretely, for an autoregressive drafter, generating γ draft tokens requires γ draft steps, each consisting of a draft-model forward computation followed by an LM-head projection. Denoting the average latency of these two operations by t_{net} and t_{head}, respectively, the total drafting cost can be approximated as

$$T_{\text{draft}}^{\text{AR}} \approx \gamma \cdot (t_{\text{net}} + t_{\text{head}}) .$$

This cost grows approximately linearly with the speculation budget γ, and can be further amplified when the draft model becomes deeper or the vocabulary size is large (Zhao et al., 2025; Yan et al., 2025). Therefore, although autoregressive drafters often achieve high acceptance lengths, their repeated executions can offset the gain from longer accepted prefixes and limit the achievable speedup.

3.3 Parallel Drafting

Unlike autoregressive drafters that factorize the draft distribution from left to right, a parallel drafter (Chen et al., 2026; Liu et al., 2026a) directly predicts the block-level conditional distribution

$$q_{\text{PAR}}(x_{t+1:t+\gamma} \mid x_{\le t}),$$

thereby generating multiple draft tokens in parallel. Its drafting cost can be approximated as

$$T_{\text{draft}}^{\text{PAR}} \approx t_{\text{net}}^{\text{block}} + t_{\text{head}}^{\text{block}},$$

where $t_{\text{net}}^{\text{block}}$ and $t_{\text{head}}^{\text{block}}$ denote the average latency of the block-level draft-model forward pass and the corresponding LM-head projection, respectively. Unlike autoregressive drafting, they are incurred once for the whole draft block rather than repeated γ times, enabling better GPU utilization through parallel computation.

The verification stage remains similar for both autoregressive and parallel drafters: the target model evaluates the proposed draft block in parallel and accepts the longest valid prefix. The main difference lies in draft generation. Autoregressive drafters usually achieve longer acceptance lengths because they explicitly model causal dependencies within the draft sequence, but their draft cost grows with the number of generated tokens. Parallel drafters reduce or eliminate this sequential drafting cost, but often weaken intra-block causal dependencies among draft tokens. Consequently, they may require larger draft capacity to reach acceptance length comparable to autoregressive drafters (Chen et al., 2026; Zhang et al., 2026).

4 Methodology

4.1 Architecture of Domino

Figure 3 gives an overview of Domino. Our method contains two components: a parallel draft backbone and a *Domino head*. The parallel draft backbone generates preliminary distributions for all positions in the draft block in a single parallel computation. The Domino head then refines these preliminary distributions by propagating causal information across draft positions.

4.1.1 Parallel Draft Backbone

We instantiate the parallel draft backbone with the DFlash architecture (Chen et al., 2026), which generates block-level draft representations from target context features and masked block inputs. Given a verified prefix $x_{\le t}$, we use the last verified token x_t as the anchor and construct a masked draft block

$$\tilde{x}_{t:t+B-1} = [x_t, \texttt{[MASK]}, \ldots, \texttt{[MASK]}],$$

where the remaining $B-1$ positions correspond to future draft tokens. The backbone takes the target-model context features C_t extracted from the verified prefix and the embeddings of the masked draft block as input, and produces block-level hidden states in one non-autoregressive forward pass:

$$H_{t:t+B-1} = \text{Backbone}\left(C_t,\, \text{Embed}(\tilde{x}_{t:t+B-1})\right).$$

The preliminary logits for future positions are computed by applying the frozen target LM head:

$$L_i^{\text{base}} = \text{LMHead}(H_i), \quad i = t{+}1, \ldots, t{+}B{-}1,$$

where LMHead denotes the LM head of the target model. The parallel backbone produces hidden representations for the entire draft block in parallel, from which base logits are computed using the target LM head.

4.1.2 Domino Head

The Domino head injects causal information into the parallel base logits. It contains a causal encoder and a low-rank correction head.

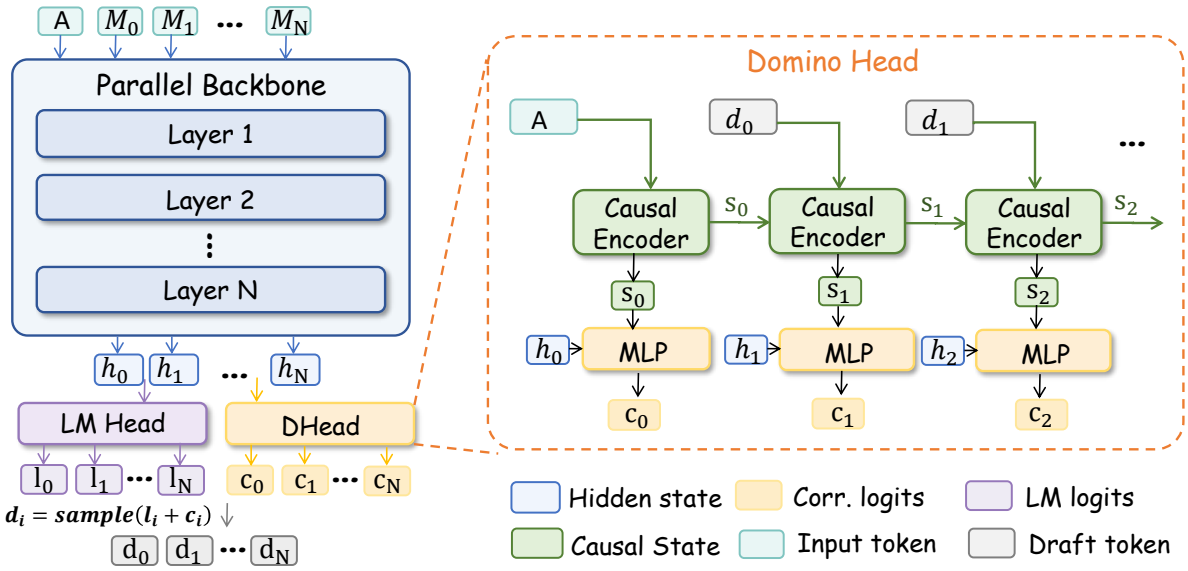


Figure 3: Overview of Domino. The parallel backbone produces hidden states for the whole draft block in one forward pass. The Domino head sequentially updates a causal state from previously sampled draft tokens and generates correction logits c_i, which refine the base logits l_i. Each draft token is sampled from the final logits $l_i + c_i$.

Causal Encoder. For each draft position i, we use a lightweight GRU to summarize the embeddings of preceding draft tokens (Cho et al., 2014). Let E_j denote the token embedding at position j. The causal state before predicting position i is

$$S_{i-1} = \mathrm{GRU}(E_{\leq i-1}),$$

where $S_{i-1} \in \mathbb{R}^{d_s}$ represents the prefix-dependent information available to position i. In practice, the GRU is lightweight; in our implementation, we use a hidden dimension of 1024. This causal state allows later draft positions to receive information from earlier draft tokens without invoking a full autoregressive draft model.

Low-Rank Correction Head. Given the base representation H_i and causal state S_{i-1}, the Domino head produces a logit-space residual correction through a low-rank bottleneck:

$$\Delta L_i = W_2\, \sigma(W_1[H_i; S_{i-1}])\,,$$

where W_1 projects the concatenated representation into a low-rank hidden space of dimension r, W_2 maps the low-rank representation to the vocabulary space, and σ is the SiLU activation. The final draft logits are then computed as

$$L_i = L_i^{\text{base}} + \Delta L_i.$$

In our implementation, $r = 256$. Since the correction is computed from a low-rank hidden space, it is much cheaper than repeatedly applying a full LM head in an autoregressive draft loop.

We apply correction in logit space rather than hidden space. A hidden-space correction would require applying the full LM head again after each causal update, reintroducing the expensive full-head computation into the sequential branch. In contrast, our logit-space correction keeps the base LM-head computation parallel and restricts the causal branch to a low-rank residual update.

4.2 Training

We describe two training choices for Domino, each addressing a different failure mode.

Teacher-Forced Causal Encoding. The causal encoder consumes the preceding draft tokens within the current block. A natural choice is to feed it self-generated prefixes during training, following the training-time testing (TTT) strategy used in EAGLE-3 (Li et al., 2025b), which simulates multi-step draft generation during training. Instead, we use teacher forcing and feed the encoder ground-truth token embeddings.

There are two reasons for this choice. First, self-generated prefixes can be noisy and often incorrect, especially early in training. Supervising the model to map such corrupted prefixes to the ground-truth next token creates an input–output mapping that does not exist in the underlying data distribution

(Huszár, 2015). This mismatch can degrade the causal representations learned by the GRU.

Second, teacher forcing is better aligned with the acceptance mechanism of speculative decoding. A draft token at position i contributes to the acceptance length only when all preceding draft tokens have already been verified as correct. Therefore, the correction at position i only matters in the accepted-prefix regime, where the preceding draft tokens match the target sequence. Training the causal encoder on ground-truth prefixes directly focuses learning on this regime; corrections conditioned on incorrect prefixes are less relevant, since those positions will be rejected by verification. We compare teacher forcing with TTT in Section 5.3.2.

Base-anchored curriculum. Teacher forcing introduces another failure mode. Since the correction branch receives clean prefixes during training, directly optimizing only the final-logit loss can allow the correction branch to shortcut the parallel backbone. In this case, the base logits may become weak, and the final prediction may rely too heavily on the correction branch. As shown in Figure 4, this leads to a collapse of the parallel backbone loss, while the proposed base-anchored curriculum keeps the backbone loss decreasing steadily.

To prevent this collapse, we jointly supervise the base and final logits with a time-varying weight:

$$\mathcal{L} = (1 - \lambda_t)\,\mathcal{L}_{\text{final}} + \lambda_t\,\mathcal{L}_{\text{base}},$$

where $\mathcal{L}_{\text{final}}$ and $\mathcal{L}_{\text{base}}$ are cross-entropy losses computed from L_i and L_i^{base}, respectively. We linearly anneal λ_t from 1 to 0 over the course of training, so the objective is initially anchored on the base logits—forcing the parallel backbone to learn a strong base distribution—and gradually shifts to the final logits as the Domino head takes over the residual correction. We ablate this curriculum against direct final-loss training in Section 5.3.2.

Following common practice in block-level speculative decoding (Chen et al., 2026), both $\mathcal{L}_{\text{base}}$ and $\mathcal{L}_{\text{final}}$ use a position-wise exponential decay $w_k = \exp(-k/\gamma)$ to prioritize earlier draft positions, whose acceptance gates the rest of the block.

4.3 Efficient Runtime Implementation

To minimize the overhead of the Domino head, we implement its correction loop with fused Triton kernels and CUDA Graphs. This reduces kernel-launch and Python-level overhead during rollout. Under the latency setting in Figure 1, the Domino-head latency decreases from 2.64ms to 1.20ms.

5 Experiments

5.1 Experimental Setup

Models and Evaluations. We evaluate Domino on Qwen3-4B, and Qwen3-8B (Qwen Team, 2025). Following the evaluation protocol of DFlash (Chen et al., 2026), we consider tasks from three categories: math reasoning, code generation, and open-ended dialogue. For math reasoning, we evaluate on GSM8K (Cobbe et al., 2021), MATH (Hendrycks et al., 2021), and AIME25 (Mathematical Association of America, 2025); for code generation, we use HumanEval (Chen et al., 2021), MBPP (Austin et al., 2021), and LiveCodeBench (Jain et al., 2024); for dialogue, we evaluate on MT-Bench (Zheng et al., 2023) and Alpaca (Taori et al., 2023). For each task, we report the average acceptance length τ and the end-to-end decoding speedup over the autoregressive baseline.

Training Data. We train the draft modules on `mlabonne/open-perfectblend`[1], a instruction-tuning dataset with 1.42M samples covering chat, math, code, and general instruction-following tasks. We regenerate all responses using the corresponding target model rather than using the original dataset responses. Additional training details are provided in Appendix A.1.

Baselines. We compare Domino with vanilla autoregressive decoding and representative speculative decoding baselines, including EAGLE-3 (Li et al., 2025b), DFlash (Chen et al., 2026), DART (Liu et al., 2026a), and FR-Spec (Zhao et al., 2025). EAGLE-3 represents autoregressive drafting; DFlash and DART represent parallel drafting methods that reduce repeated per-token draft computation; and FR-Spec represents vocabulary-efficient speculative decoding by reducing full-vocabulary LM-head projection cost.

Implementation Details. For Domino, we use a draft block size of 16 for all target models and a 5-layer parallel draft backbone. The hidden dimension of the GRU causal encoder is set to 1024, and the hidden dimension of the low-rank correction head is set to 256. Unless otherwise specified, all experiments are conducted on NVIDIA A100-SXM4-80GB GPUs.

[1] https://huggingface.co/datasets/mlabonne/open-perfectblend

Table 1: Decoding speedup over vanilla autoregressive decoding and average acceptance length (τ) on Qwen3 models with a maximum of 2048 generated tokens. Parenthesized values indicate the draft tree size for EAGLE-3 and DART, and the draft block size for DFlash and DOMINO. The average is computed over all listed benchmarks. For EAGLE-3, DFlash, and DART, we use the publicly released checkpoints listed in Table 5.

Model	Method	MATH						CODE						CHAT				OVERALL	
		GSM8K		MATH-500		AIME25		HumanEval		MBPP		LCB		MT-Bench		Alpaca		*Avg.*	
Temperature = 0		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3-4B	EAGLE-3 (16)	2.24×	3.32	2.10×	3.11	2.08×	3.10	2.09×	3.09	2.02×	2.99	1.95×	2.93	1.95×	2.94	1.87×	2.83	2.04×	3.04
	EAGLE-3 (60)	2.57×	3.83	2.40×	3.59	2.36×	3.53	2.36×	3.53	2.30×	3.44	2.19×	3.31	2.25×	3.41	2.14×	3.29	2.32×	3.49
	DART (60)	2.16×	2.65	2.20×	2.63	2.13×	2.59	2.48×	2.99	2.47×	3.01	2.23×	2.76	2.19×	2.69	2.16×	2.78	2.25×	2.76
	DFlash (16)	5.18×	6.51	6.10×	7.79	5.87×	7.47	5.23×	6.63	4.77×	6.12	**5.48×**	**7.15**	2.72×	4.12	2.21×	3.12	4.70×	6.11
	DOMINO (16)	**8.02×**	**10.12**	**7.14×**	**9.01**	**5.97×**	**7.49**	**5.66×**	**7.02**	**5.59×**	**7.11**	5.33×	6.86	**3.28×**	**5.12**	**2.80×**	**3.88**	**5.47×**	**7.08**
Qwen3-8B	EAGLE-3 (16)	2.21×	3.27	2.09×	3.10	2.07×	3.07	2.17×	3.21	1.93×	2.86	1.80×	2.95	1.82×	2.75	1.67×	2.53	1.97×	2.97
	EAGLE-3 (60)	2.56×	3.80	2.42×	3.61	2.41×	3.59	2.50×	3.74	2.22×	3.31	2.03×	3.12	2.07×	3.17	1.88×	2.90	2.26×	3.41
	DART (60)	2.28×	2.71	2.29×	2.70	2.11×	2.59	2.52×	2.95	2.39×	2.98	2.24×	2.78	2.27×	3.03	2.21×	2.84	2.29×	2.82
	DFlash (16)	5.21×	6.59	6.18×	7.87	5.67×	7.13	5.21×	6.52	4.71×	5.98	**5.37×**	**7.12**	2.73×	4.18	2.23×	3.10	4.66×	6.06
	DOMINO (16)	**7.92×**	**10.03**	**7.38×**	**9.43**	**5.85×**	**7.41**	**5.89×**	**7.39**	**5.53×**	**7.04**	5.27×	7.04	**3.29×**	**5.18**	**2.78×**	**3.87**	**5.49×**	**7.17**
Temperature = 1		Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ	Speedup	τ
Qwen3-4B	EAGLE-3 (16)	2.18×	3.26	2.00×	3.00	1.86×	2.80	2.02×	3.04	1.97×	2.95	1.87×	2.83	1.90×	2.91	1.78×	2.72	1.95×	2.94
	EAGLE-3 (60)	2.44×	3.77	2.28×	3.51	2.08×	3.10	2.09×	3.09	2.19×	3.38	2.05×	3.19	2.11×	3.33	2.02×	3.17	2.16×	3.32
	DART (60)	2.19×	2.65	2.20×	2.63	2.13×	2.59	2.48×	2.99	2.46×	3.01	2.23×	2.76	2.19×	2.69	2.16×	2.78	2.26×	2.76
	DFlash (16)	4.66×	5.98	5.02×	6.58	**3.79×**	**5.05**	4.73×	5.96	4.44×	5.67	4.76×	6.38	2.62×	3.97	2.18×	3.03	4.03×	5.33
	DOMINO (16)	**6.79×**	**8.61**	**5.67×**	**7.35**	3.75×	4.83	**5.02×**	**6.30**	**4.96×**	**6.36**	**5.04×**	**6.45**	**3.02×**	**4.56**	**2.59×**	**3.52**	**4.61×**	**6.00**
Qwen3-8B	EAGLE-3 (16)	2.18×	3.26	1.96×	3.00	1.86×	2.80	2.05×	3.09	1.97×	2.95	1.90×	2.93	1.91×	2.91	1.78×	2.83	1.95×	2.97
	EAGLE-3 (60)	2.40×	3.70	2.23×	3.44	2.05×	3.17	2.32×	3.58	2.11×	3.24	1.90×	2.95	1.91×	3.02	1.78×	2.81	2.09×	3.24
	DART (60)	2.25×	2.71	2.25×	2.70	2.13×	2.59	2.45×	2.95	2.45×	2.98	2.25×	2.78	2.24×	2.73	2.18×	2.84	2.28×	2.79
	DFlash (16)	4.71×	6.00	5.00×	6.50	3.72×	4.77	4.34×	5.44	4.14×	5.23	5.04×	**6.73**	2.55×	3.87	2.16×	2.93	3.96×	5.18
	DOMINO (16)	**6.47×**	**8.34**	**5.40×**	**7.20**	**3.78×**	**4.92**	**4.75×**	**5.98**	**4.73×**	**6.02**	**5.06×**	6.72	**2.94×**	**4.61**	**2.52×**	**3.46**	**4.46×**	**5.91**

5.2 Main Results

Low-concurrency case. Table 1 reports the end-to-end speedup and average acceptance length on Qwen3 models under the Transformers backend. Domino consistently outperforms autoregressive drafting methods such as EAGLE-3, as well as parallel drafting baselines including DART and DFlash. Compared with DFlash, Domino further improves the average speedup from $4.70\times$ to $5.47\times$ on Qwen3-4B and from $4.66\times$ to $5.49\times$ on Qwen3-8B under greedy decoding ($T = 0$). Similar gains hold under sampling decoding ($T = 1$), where the average speedup increases from $4.03\times$ to $4.61\times$ on Qwen3-4B and from $3.96\times$ to $4.46\times$ on Qwen3-8B. These results show that causal correction improves draft quality with little additional overhead, leading to a higher end-to-end speedup.

High-concurrency case. We further evaluate serving throughput under different concurrency levels using SGLang. As shown in Table 2, Domino achieves higher throughput than EAGLE-3 and DFlash on both Qwen3-4B and Qwen3-8B. The gains indicate that the improved draft quality of Domino can be effectively translated into practical serving throughput, while maintaining the low-overhead advantage of block-parallel drafting.

5.3 Ablation

We conduct ablation studies to understand the source of Domino's improvement. Specifically, we examine whether the gain comes from differences in training data, the proposed training strategy, and the lightweight Domino head. Unless otherwise specified, all ablation experiments are conducted on Qwen3-8B with greedy decoding. The draft models are trained on ShareGPT, and evaluations are performed on NVIDIA A100 GPUs.

5.3.1 Training Data

To isolate the impact of model architecture, we train all baselines on identical data. During evaluation, we apply greedy decoding with a fixed 16-token drafting budget across all methods.

Table 3 shows a clear trade-off between acceptance length and drafting overhead. EAGLE-3 achieves strong acceptance lengths, e.g., 5.01 on GSM8K, but its throughput is limited by sequential drafting. DFlash has lower acceptance lengths, e.g., 3.90 on GSM8K and 3.78 on HumanEval, but obtains higher throughput by using parallel drafting. Domino achieves a better balance: it reaches comparable acceptance lengths to autoregressive methods while maintaining low drafting overhead, resulting in the best throughput on almost all tasks and concurrency levels. This confirms that the gain comes from the proposed design rather than

Table 2: High-concurrency throughput on Qwen3 models. Baseline rows report absolute throughput in TPS. Other entries report TPS with green subscripts indicating speedup over the corresponding baseline.

Task	Method	Concurrency				
		2	4	8	16	32
Qwen3-4B						
GSM8K	Baseline	293	557	1079	1884	2868
	EAGLE-3 (16)	$453_{1.5\times}$	$832_{1.5\times}$	$1375_{1.3\times}$	$1839_{1.0\times}$	$2170_{0.8\times}$
	EAGLE-3 (60)	$458_{1.6\times}$	$683_{1.2\times}$	$896_{0.8\times}$	$1040_{0.6\times}$	$1133_{0.4\times}$
	DFlash (16)	$965_{3.3\times}$	$1698_{3.0\times}$	$2738_{2.5\times}$	$3538_{1.9\times}$	$4397_{1.5\times}$
	DOMINO (16)	$\mathbf{1256}_{4.3\times}$	$\mathbf{2202}_{4.0\times}$	$\mathbf{3441}_{3.2\times}$	$\mathbf{4467}_{2.4\times}$	$\mathbf{5509}_{1.9\times}$
MBPP	Baseline	291	544	1002	1586	2300
	EAGLE-3 (16)	$407_{1.4\times}$	$740_{1.4\times}$	$1186_{1.2\times}$	$1597_{1.0\times}$	$1892_{0.8\times}$
	EAGLE-3 (60)	$414_{1.4\times}$	$610_{1.1\times}$	$793_{0.8\times}$	$899_{0.6\times}$	$987_{0.4\times}$
	DFlash (16)	$914_{3.1\times}$	$1650_{3.0\times}$	$2501_{2.5\times}$	$3330_{2.1\times}$	$4088_{1.8\times}$
	DOMINO (16)	$\mathbf{968}_{3.3\times}$	$\mathbf{1654}_{3.0\times}$	$\mathbf{2651}_{2.6\times}$	$\mathbf{3422}_{2.2\times}$	$\mathbf{4290}_{1.9\times}$
Qwen3-8B						
GSM8K	Baseline	184	360	655	1143	1713
	EAGLE-3 (16)	$324_{1.8\times}$	$598_{1.7\times}$	$979_{1.5\times}$	$1276_{1.1\times}$	$1398_{0.8\times}$
	EAGLE-3 (60)	$330_{1.8\times}$	$482_{1.3\times}$	$577_{0.9\times}$	$624_{0.5\times}$	$699_{0.4\times}$
	DFlash (16)	$672_{3.7\times}$	$1243_{3.4\times}$	$1915_{2.9\times}$	$2533_{2.2\times}$	$2801_{1.6\times}$
	DOMINO (16)	$\mathbf{942}_{5.1\times}$	$\mathbf{1703}_{4.7\times}$	$\mathbf{2678}_{4.1\times}$	$\mathbf{3379}_{3.0\times}$	$\mathbf{3650}_{2.1\times}$
MBPP	Baseline	183	352	635	1076	1428
	EAGLE-3 (16)	$290_{1.6\times}$	$525_{1.5\times}$	$886_{1.4\times}$	$1180_{1.1\times}$	$1291_{0.9\times}$
	EAGLE-3 (60)	$293_{1.6\times}$	$438_{1.2\times}$	$525_{0.8\times}$	$574_{0.5\times}$	$631_{0.4\times}$
	DFlash (16)	$649_{3.6\times}$	$1169_{3.3\times}$	$1889_{3.0\times}$	$2487_{2.3\times}$	$2800_{2.0\times}$
	DOMINO (16)	$\mathbf{701}_{3.8\times}$	$\mathbf{1256}_{3.6\times}$	$\mathbf{2035}_{3.2\times}$	$\mathbf{2727}_{2.5\times}$	$\mathbf{3027}_{2.1\times}$

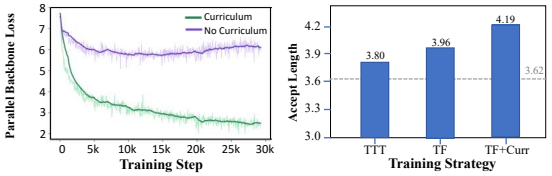


Figure 4: Left: parallel backbone loss with and without the base-anchored curriculum. Right: average acceptance length under TTT, TF, and TF+Curr. TTT denotes training-time testing, TF denotes teacher forcing, and Curr denotes the base-anchored curriculum. The gray dashed line denotes the DFlash reference.

training-data differences.

5.3.2 Training Strategy

We ablate the training strategy for the causal correction branch in Figure 4. Compared with training-time test (TTT), teacher forcing improves the average acceptance length from 3.80 to 3.96. This supports our motivation that the causal encoder should be trained on ground-truth prefixes rather than noisy self-generated prefixes, since only draft positions whose previous tokens have been accepted can contribute to the final acceptance length.

However, direct teacher forcing alone is not sufficient. As discussed in Section 4.2, clean ground-truth prefixes can allow the correction branch to shortcut the parallel backbone, causing the base logits to collapse. This is reflected in Figure 4, where direct final-logit training keeps the parallel backbone loss high. The base-anchored curriculum mitigates this issue by first strengthening the base logits and then gradually shifting optimization toward the final logits. As a result, TF+Curriculum further improves the average acceptance length from 3.96 to 4.19. These results validate both parts of our training strategy: teacher forcing provides a more useful learning signal for causal correction, while the base-anchored curriculum prevents backbone collapse and improves the final draft quality.

Table 3: Same-data comparison on Qwen3-8B under greedy decoding. All draft models are trained on ShareGPT with the same 16-token drafting budget. FR-Spec uses a 32K frequency-ranked vocabulary subset from ShareGPT. Baseline reports TPS; other methods report speedup over the baseline.

Method	Concurrency						*Avg.* τ
	1	2	4	8	16	32	
GSM8K							
Baseline (TPS)	92	184	360	655	1143	1713	–
EAGLE-3	2.35×	2.15×	1.90×	1.77×	1.30×	0.97×	5.01
FR-Spec	2.77×	2.52×	2.36×	2.09×	1.53×	1.16×	4.79
DFlash	2.68×	2.40×	2.30×	1.95×	1.49×	1.09×	3.90
DOMINO	**3.01**×	**2.70**×	**2.61**×	**2.18**×	**1.68**×	**1.24**×	4.65
HumanEval							
Baseline (TPS)	92	183	355	667	1211	2036	–
EAGLE-3	2.27×	2.13×	2.00×	1.74×	1.27×	0.87×	4.84
FR-Spec	2.67×	2.43×	2.30×	2.03×	1.50×	1.02×	4.54
DFlash	2.58×	2.43×	2.32×	2.05×	1.47×	1.00×	3.78
DOMINO	**2.82**×	**2.64**×	**2.52**×	**2.23**×	**1.63**×	**1.12**×	4.35
LiveCodeBench							
Baseline (TPS)	91	175	283	569	744	1140	–
EAGLE-3	1.99×	1.84×	1.81×	1.41×	1.30×	0.92×	4.38
FR-Spec	2.36×	2.18×	2.17×	1.64×	1.51×	1.10×	4.21
DFlash	2.36×	2.15×	2.16×	1.56×	1.56×	1.10×	3.66
DOMINO	**2.55**×	**2.33**×	**2.44**×	**1.75**×	**1.60**×	**1.17**×	4.24

Table 4: Effect of the Domino head. We report average results here and provide full results in Table 6.

Method	Avg. Accept Length	Avg. Speedup
w/o Domino Head	3.49	2.84×
w/ Domino Head	**4.19**	**3.31**×

5.3.3 Effect of Domino Head

We ablate the Domino head by evaluating the same model with the causal correction branch disabled or enabled. As shown in Table 4, enabling the Domino head improves the average acceptance length from 3.49 to 4.19 and the average speedup from 2.84× to 3.31×. This confirms that lightweight prefix-dependent correction is the key source of the improvement over the parallel backbone alone.

6 Conclusion

We propose Domino, a speculative decoding framework that improves block-parallel drafting with lightweight causal correction. Domino decouples

causal dependency from expensive autoregressive execution, improving draft quality while maintaining low drafting overhead. Experiments on Qwen3 models demonstrate consistent gains in acceptance length and end-to-end speedup over representative baselines. These results suggest that causal information can be effectively reintroduced into parallel drafting for faster LLM inference.

7 Limitations

This work focuses on inference acceleration rather than reducing the cost of model training or finetuning. Our current implementation is mainly adapted to SGLang, and its compatibility with other serving frameworks remains to be systematically evaluated. Moreover, the practical speedup can vary across hardware platforms due to differences in memory bandwidth, compute capability, and kernel efficiency. As a result, further platform-specific optimization may be needed for deployment in different environments.

References

Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. 2026. Pard: Accelerating llm inference with low-cost parallel draft model adaptation. In *International Conference on Learning Representations*.

Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. 2024. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 5209–5235. PMLR.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*.

Jian Chen, Yesheng Liang, and Zhijian Liu. 2026. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, pages 1724–1734.

Jacob K Christopher, Brian R Bartoldson, Tal Ben-Nun, Michael Cardei, Bhavya Kailkhura, and Ferdinando Fioretto. 2025. Speculative diffusion decoding: Accelerating language generation through diffusion. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 12042–12059.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro

Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Zhenyu He, Zexuan Zhong, Tianle Cai, Jason Lee, and Di He. 2024. Rest: Retrieval-based speculative decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1582–1595.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.

Ferenc Huszár. 2015. How (not) to train your generative model: Scheduled sampling, likelihood, adversary? *arXiv preprint arXiv:1511.05101*.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2024. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR.

Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. 2025a. Diffuspec: Unlocking diffusion language models for speculative decoding. *arXiv preprint arXiv:2510.02358*.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024a. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 7421–7432. Association for Computational Linguistics.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024b. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 28935–28948. PMLR.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2025b. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *Advances in Neural Information Processing Systems*.

Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyan Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. 2026a. Dart: Diffusion-inspired speculative decoding for fast llm inference. *arXiv preprint arXiv:2601.19278*.

Runheng Liu, Jincheng Xie, Wen Hu, Xingchen Xiao, and Heyan Huang. 2026b. Adapld: Adaptive retrieval and reuse for efficient model-free speculative decoding. *Preprint*, arXiv:2606.05742.

Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. 2025. Pearl: Parallel speculative decoding with adaptive draft length. In *International Conference on Learning Representations*, volume 2025, pages 1085–1104.

Mathematical Association of America. 2025. American invitational mathematics examination 2025. American Mathematics Competitions. AIME 2025 problems.

Bradley McDanel, Steven Li, Sruthikesh Surineni, and Harshit Khaitan. 2026. Moe-spec: Expert budgeting for efficient speculative decoding. *arXiv preprint arXiv:2602.16052*.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, and 1 others. 2023. Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*.

Qwen Team. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, and 1 others. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models*. Https://crfm.stanford.edu/2023/03/13/alpaca.html.

Miles Williams, Young D Kwon, Rui Li, Alexandros Kouris, and Stylianos I Venieris. 2026. Speculative decoding with a speculative vocabulary. *arXiv preprint arXiv:2602.13836*.

Jincheng Xie, Yawen Ling, Qi Xiao, Feiyu Zhang, Zhongyi Huang, Wen Hu, and Yu Zheng. 2026a. Spectre: Hybrid ordinary-parallel speculative serving for resource-efficient llm inference. *Preprint*, arXiv:2605.08151.

Jincheng Xie, Runheng Liu, Heyan Huang, Yawen Ling, Hanbin Dai, Yu Zheng, and Wen Hu. 2026b. Less experts, faster decoding: Cost-aware speculative decoding for mixture-of-experts. *Preprint*, arXiv:2607.12696.

Siyuan Yan, Mo Zhu, Guo-qing Jiang, Jianfei Wang, Jiaxing Chen, Wentai Zhang, Xiang Liao, Xiao Cui, Chen Zhang, Zhuoran Song, and 1 others. 2025. Scaling laws for speculative decoding. *arXiv preprint arXiv:2505.07858*.

Yaojie Zhang, Jianuo Huang, Junlong Ke, Yuhang Han, Yongji Long, Tianchen Zhao, Biqing Qi, and Linfeng Zhang. 2026. Flexdraft: Flexible speculative decoding via attention tuning and bonus-guided calibration. *arXiv preprint arXiv:2605.20022*.

Weilin Zhao, Tengyu Pan, Xu Han, Yudi Zhang, Sun Ao, Yuxiang Huang, Kaihuo Zhang, Weilun Zhao, Yuxuan Li, Jie Zhou, and 1 others. 2025. Fr-spec: Accelerating large-vocabulary language models via frequency-ranked speculative sampling. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3909–3921.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems*.

A Appendix

A.1 Training Details

For both Qwen3-4B and Qwen3-8B, we train the DOMINOdraft module while keeping the target model frozen. We use the regenerated PerfectBlend data described in Section 5. Input sequences are truncated to a maximum length of 3072 tokens, and the draft block size is set to 16.

Unless otherwise specified, all draft modules are trained for 3 epochs on 8 NVIDIA A100-SXM4-80GB GPUs. We use a per-GPU batch size of 2, resulting in a global batch size of 16 without gradient accumulation. We optimize the model with AdamW using a learning rate of 6×10^{-4}, zero weight decay, gradient clipping with a maximum norm of 1.0, and a cosine learning-rate schedule with a warmup ratio of 0.04. Training is conducted in bfloat16 precision with FSDP and gradient sharding.

Table 5: Baseline draft model checkpoints used in our experiments.

Method	**Qwen3-4B**	**Qwen3-8B**
EAGLE-3	`Angslim/Qwen3-4B_eagle3`	`Angslim/Qwen3-8B_eagle3`
DART	`fvliang/qwen4b-dart`	`fvliang/qwen8b-dart`
DFlash	`z-lab/Qwen3-4B-DFlash-b16`	`z-lab/Qwen3-8B-DFlash-b16`

Table 6: Full benchmark-level results for the Domino head ablation. The same trained model is evaluated with the causal correction branch disabled or enabled.

Benchmark	w/o Domino Head		w/ Domino Head	
	Accept Length	Speedup	Accept Length	Speedup
GSM8K	3.82	3.17×	**4.80**	**3.84**×
MATH-500	3.76	3.08×	**4.66**	**3.74**×
AIME25	3.68	3.03×	**4.29**	**3.47**×
HumanEval	3.69	3.04×	**4.35**	**3.51**×
MBPP	3.30	2.70×	**3.94**	**3.16**×
LiveCodeBench	3.36	2.64×	**3.92**	**2.98**×
MT-Bench	2.84	2.21×	**3.39**	**2.46**×
Avg.	3.49	2.84×	**4.19**	**3.31**×